

Sistemas de Numeração e Erros em Processos Numéricos

Cálculo Numérico – EXA1132

Prof. Paulo César Linhares da Silva

Universidade Federal Rural do Semi-Árido – UFERSA
Bacharelado em Ciência da Computação

2026.2

Por que estudar isso em Ciência da Computação?

O computador não "pensa" em decimal

Toda informação processada por um computador – números, texto, imagens, som – é, no nível mais baixo, representada por sequências de dígitos binários (*bits*).

- Entender sistemas de numeração é entender **como a máquina representa a informação**;
- Entender erros numéricos é entender **os limites dessa representação**;
- Sem essa base, é impossível analisar corretamente algoritmos numéricos, depurar bugs sutis de precisão ou avaliar a confiabilidade de um cálculo científico.

Pergunta-guia da aula

Se um computador só entende 0 e 1, como ele representa $-17,5$? E por que $0,1 + 0,2 \neq 0,3$ em Python?

Sistemas posicionais de numeração

Definição (Sistema posicional)

Um sistema de numeração é **posicional** quando o valor de cada dígito depende da posição que ele ocupa. Dado uma base $b \in \mathbb{N}$, $b \geq 2$, todo número natural pode ser escrito como combinação de potências de b , com dígitos $a_i \in \{0, 1, \dots, b - 1\}$.

Sistema	Base	Dígitos	Uso típico
Decimal	10	0–9	Notação humana
Binário	2	0, 1	Hardware, lógica digital
Octal	8	0–7	Permissões Unix, sistemas antigos
Hexadecimal	16	0–9, A–F	Endereços de memória, cores, hashes

Notação posicional geral

Um número N na base b , com $n + 1$ dígitos, é escrito como

$$N = (a_n a_{n-1} \cdots a_1 a_0)_b$$

e seu valor em termos de potências de b é dado por

$$N = a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b^1 + a_0 b^0 = \sum_{i=0}^n a_i b^i.$$

Exemplo

$$(1101)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = (13)_{10}$$

$$(2F)_{16} = 2 \cdot 16^1 + 15 \cdot 16^0 = 32 + 15 = (47)_{10}$$

Sistema decimal e sistema binário

Decimal (base 10)

- Base natural da contagem humana (10 dedos);
- Dígitos: 0, 1, ..., 9;
- Cada posição vale uma potência de 10.

Binário (base 2)

- Base nativa dos circuitos digitais (dois estados: aceso/apagado, alta/baixa tensão);
- Dígitos: 0, 1 (*bit*);
- 8 bits = 1 *byte*.

Por que o computador usa base 2, e não base 10?

Um transistor é naturalmente biestável (ligado/desligado). Construir circuitos confiáveis com 10 níveis de tensão distintos é muito mais caro e sujeito a ruído do que distinguir apenas dois níveis.

Sistema octal e sistema hexadecimal

Octal (base 8)

- Dígitos: 0, 1, ..., 7;
- $8 = 2^3$: cada dígito octal representa **3 bits**;
- Usado historicamente em minicomputadores e em permissões de arquivo no Unix/Linux (ex.: `chmod 755`).

Hexadecimal (base 16)

- Dígitos: 0–9 e *A, B, C, D, E, F* (10 a 15);
- $16 = 2^4$: cada dígito hexadecimal representa **4 bits** (um *nibble*);
- Muito mais compacto que binário para representar endereços de memória, cores e hashes.

Exemplo

$$(11111111)_2 = (377)_8 = (FF)_{16} = (255)_{10}$$

Teorema da mudança de base

Teorema (Representação posicional – existência e unicidade)

Seja $b \in \mathbb{N}$, $b \geq 2$. Todo número natural $N \geq 0$ admite uma **única** representação da forma

$$N = \sum_{i=0}^n a_i b^i, \quad 0 \leq a_i \leq b-1, \quad a_n \neq 0 \text{ (se } N > 0 \text{)}.$$

Ideia da prova (existência): pelo *algoritmo da divisão* de Euclides, para todo N existem únicos q_0, a_0 tais que

$$N = q_0 b + a_0, \quad 0 \leq a_0 < b.$$

Repetindo o processo para q_0 , obtendo q_1, a_1 , e assim sucessivamente, até que $q_n = 0$:

$$q_0 = q_1 b + a_1, \quad q_1 = q_2 b + a_2, \quad \dots, \quad q_{n-1} = 0 \cdot b + a_n.$$

Como q_i é estritamente decrescente e limitado inferiormente por 0, o processo termina (princípio da boa ordenação de $\mathbb{N}$), o que garante a **existência**. A **unicidade** decorre da unicidade do quociente e resto no algoritmo da divisão em cada etapa.

Corolário: o algoritmo de mudança de base

Corolário (Conversão de decimal para a base b)

*Os dígitos $a_0, a_1, \dots, a_n$ da representação de N na base b são obtidos pelas divisões inteiras sucessivas de N por b , lendo-se os **restos** do último para o primeiro (do mais significativo ao menos significativo):*

$$N = q_0 b + a_0, \quad q_0 = q_1 b + a_1, \quad \dots, \quad q_{n-1} = 0 \cdot b + a_n.$$

Exemplo (Convertendo $N = (45)_{10}$ para binário)

$$45 = 22 \cdot 2 + 1 \quad (a_0 = 1)$$

$$22 = 11 \cdot 2 + 0 \quad (a_1 = 0)$$

$$11 = 5 \cdot 2 + 1 \quad (a_2 = 1)$$

$$5 = 2 \cdot 2 + 1 \quad (a_3 = 1)$$

$$2 = 1 \cdot 2 + 0 \quad (a_4 = 0)$$

$$1 = 0 \cdot 2 + 1 \quad (a_5 = 1)$$

Logo $(45)_{10} = (101101)_2$ (lendo os restos de baixo para cima).

Conversões entre bases: resumo prático

1) De qualquer base b para decimal

Aplicar diretamente a expansão polinomial $N = \sum_i a_i b^i$ (Teorema).

2) De decimal para qualquer base b

Divisões inteiras sucessivas por b ; os restos, lidos de trás para frente, formam os dígitos (Corolário).

3) Entre binário, octal e hexadecimal (conversão direta)

Como $8 = 2^3$ e $16 = 2^4$, basta agrupar os bits em grupos de 3 (octal) ou 4 (hexadecimal), sem passar por decimal:

$$\underbrace{10}_2 \underbrace{111}_7 \underbrace{101}_5 {}_2 \longrightarrow (275)_8$$

$$\underbrace{1010}_A \underbrace{1111}_F {}_2 \longrightarrow (AF)_{16}$$

Números fracionários em outras bases

A parte fracionária também é posicional, usando potências **negativas** de b :

$$(a_{-1}a_{-2}\cdots)_b = a_{-1}b^{-1} + a_{-2}b^{-2} + \cdots$$

Exemplo

$$(0,101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0,5 + 0,125 = (0,625)_{10}$$

Observação importante

Nem toda fração decimal finita tem representação binária **finita**. Por exemplo, $(0,1)_{10}$ é uma dízima em binário: $(0,0001100110011\dots)_2$. Essa é a raiz de muitos erros de arredondamento no computador!

Por que surgem erros?

- Um computador representa números reais com uma quantidade **finita** de bits;
- Como acabamos de ver, muitos números não têm representação binária finita;
- Logo, ao armazenar 0,1 na memória, o computador guarda uma **aproximação**;
- Cada operação aritmética pode introduzir e **propagar** novos erros.

Consequência prática

Em Python: `0.1 + 0.2 == 0.3` retorna `False`. Não é um bug de linguagem – é uma consequência inevitável da representação finita em ponto flutuante (padrão IEEE 754).

Erro absoluto e erro relativo

Definição (Erro absoluto)

Se x^* é o valor aproximado de um valor exato x , o erro absoluto é

$$E_a = |x - x^*|.$$

Definição (Erro relativo)

$$E_r = \frac{|x - x^*|}{|x|}, \quad x \neq 0.$$

Exemplo

Medir 10,000 km com erro absoluto de 1 m é excelente ($E_r = 10^{-4}$); medir 1 m com o mesmo erro absoluto de 1 m é péssimo ($E_r = 1$). O erro relativo mede a **qualidade** da aproximação, não apenas sua magnitude.

Arredondamento e truncamento

Truncamento

- Descarta os dígitos além de uma certa posição;
- Simples, porém sempre *subestima* ou *superestima* na mesma direção;
- Ex.: $\pi = 3,14159 \dots \rightarrow 3,1415$

Arredondamento

- Escolhe o valor representável mais próximo;
- Erro médio tende a ser menor;
- Ex.: $\pi \rightarrow 3,1416$

Ponto flutuante (IEEE 754)

Números reais são representados como $\pm m \times 2^e$, com mantissa m e expoente e de tamanho fixo (32 bits em float, 64 bits em double). Isso limita a quantidade de dígitos significativos representáveis.

Propagação de erros e cancelamento catastrófico

Propagação de erros

Ao encadear operações aritméticas, pequenos erros de arredondamento em cada etapa podem se **acumular** e crescer ao longo de um algoritmo – especialmente em laços com muitas iterações.

Cancelamento catastrófico

Ocorre ao subtrair dois números muito próximos, com muitos dígitos significativos em comum: os dígitos corretos se cancelam e o que sobra é dominado pelo erro de arredondamento.

Exemplo

*Com precisão de 6 dígitos: $x = 1,000001$ e $y = 1,000000$. Se ambos foram calculados com um pequeno erro de arredondamento na 6ª casa, $x - y$ tem **erro relativo enorme**, mesmo que x e y individualmente estivessem bem aproximados.*

Aplicações: representação e organização de dados

- **Endereçamento de memória:** endereços de RAM e registradores são naturalmente expressos em hexadecimal (compacto e alinhado a bytes);
- **Depuração de baixo nível:** debuggers e disassemblers (gdb, objdump) exibem valores em hexadecimal;
- **Permissões de arquivo no Unix/Linux:** `chmod 755` usa a base octal, pois cada dígito codifica exatamente 3 bits de permissão (leitura/escrita/execução);
- **Cores em computação gráfica e web:** o padrão #RRGGBB usa dois dígitos hexadecimais por canal de cor (0 a 255);
- **Redes de computadores:** endereços MAC e, em parte, endereços IPv6 são representados em hexadecimal.

Aplicações: criptografia, hashing e compactação

- **Funções hash** (MD5, SHA-256) produzem saídas exibidas em hexadecimal – compacto e fácil de comparar;
- **Criptografia:** chaves e blocos cifrados são manipulados como sequências de bits, frequentemente representadas em hexadecimal para leitura humana;
- **Codificação de dados:** Base64, UTF-8 e compressão de arquivos dependem diretamente de manipulação binária eficiente;
- **Operações bit a bit** (AND, OR, XOR, *shifts*) são a base de algoritmos de baixo nível, otimização de código e estruturas como *bitmasks* e *bloom filters*.

Aplicações: erros numéricos em sistemas reais

- **Machine Learning:** problemas de *overflow/underflow* numérico ao treinar redes neurais (ex.: função softmax sem estabilização numérica);
- **Computação científica:** simulações de longa duração (clima, engenharia, física) podem divergir devido ao acúmulo de erro de arredondamento;
- **Sistemas financeiros:** cálculos monetários evitam ponto flutuante binário (usam tipos decimais de precisão fixa) justamente para não acumular erros de centavos;
- **Análise de algoritmos numéricos:** escolher entre métodos matematicamente equivalentes, mas numericamente mais ou menos **estáveis**, é uma decisão central em Cálculo Numérico.

Estudo de caso: quando o erro numérico custa caro

Patriot Missile Failure (1991, Guerra do Golfo)

O sistema de defesa antimísseis Patriot convertia o tempo, medido em décimos de segundo, usando um registrador de 24 bits. A constante 0,1 não tem representação binária finita (como vimos!), gerando um pequeno erro de truncamento. Após rodar continuamente por cerca de 100 horas, esse erro acumulado causou um desvio de tempo de aproximadamente 0,34 s – suficiente para que o sistema perdesse o rastreamento de um míssil Scud, que atingiu um alojamento militar.

A lição para Ciência da Computação

Erro numérico não é "só teoria de sala de aula": é um problema real de engenharia de software, com consequências que vão de bugs sutis a falhas catastróficas em sistemas críticos.

- 1 Sistemas de numeração são **posicionais**: o valor de cada dígito depende de sua posição e da base b ;
- 2 O **Teorema da Mudança de Base** garante existência e unicidade da representação de qualquer número em qualquer base $b \geq 2$;
- 3 Binário, octal e hexadecimal são as bases centrais em Ciência da Computação, ligadas por relações diretas com potências de 2;
- 4 A representação finita de números no computador é fonte inevitável de **erro** (absoluto, relativo, arredondamento, truncamento, cancelamento catastrófico);
- 5 Esses conceitos não são abstratos: sustentam desde endereçamento de memória e criptografia até a confiabilidade de sistemas críticos.

Zeros de funções reais: isolamento de raízes e métodos de bisseção, ponto fixo, Newton-Raphson e secante.

Bibliografia: RUGGIERO & LOPES (1996); BURDEN, FAIRES & BURDEN (2016); CHAPRA & CANALE (2016).